When Coding Agents Repeat: Persistence, Selection, and the Limits of Loop Breaking

A study of repeated tool calls in Gemma 4's coding agent

Jiarui Zha, zha.j@northeastern.edu

Abstract

We study why Gemma 4 31B, as a coding agent with thinking off, sends the same tool call again and again until its budget runs out. On 109 tasks in the harness of Kaggle's Gemma 4 Developer Agent competition, at least half the calls are repeats or malformed in a quarter of task runs, and these runs resolve 20% of the time, against 37%. Using one-step replays at fixed histories and whole-task runs under three levers, we find that: (1) the chance of one more identical call rises from 78% after two calls to 99% after ten even when a proxy holds the request fixed, so most of the rise is selection of states where the model is already near certain of the call, and identical copies in context add to it; (2) loops can be broken but not removed: where the model, free to think, chose not to, a forced thought lets the next call escape 58% of the time, against 11%; and (3) no lever measurably changes resolution.

1 Introduction

Coding agents often send the same tool call again and again until the budget runs out. In SWE-Gym, 29–40% of zero-shot runs of open models repeat one action three times in a row (Pan et al., 2025), and harnesses such as OpenHands and Gemini CLI ship loop detectors. We met this failure while building an agent with Gemma 4 31B, thinking off, for Kaggle's Gemma 4 Developer Agent competition: in one early run, half the failed tasks (23 of 46) spent at least a fifth of their calls on repeats.

Self-reinforcing repetition is known in text generation (Holtzman et al., 2020; Xu et al., 2022). After two identical actions, agents take a third 37% of the time (Liu et al., 2026), and branches of a research agent restarted with a fresh context sometimes land in a new "idea basin", which their score then selects (Cai et al., 2026). We ask whether the chance of a repeat rises because of the copies or because runs start where the model is already near certain of the call, and whether breaking loops over whole tasks, with no grader to choose among the escapes, solves more tasks.

We answer with one-step replays at recorded histories and with whole-task runs of three levers against a control. Our contributions are:

(1) Lock-in is mostly selection. With a proxy that holds the request fixed, the chance of one more identical call still rises from 78% to 99%, and identical copies add to it. Gemma 4 12B, without the proxy, locks in at the same rates (§4).

(2) Loops end from outside: at T = 0.2 the agent never ended a run of five or more identical calls itself, in 1,342 chances (§5).

(3) The pattern of the context cues the copy, not what the results say; a forced thought breaks it where the model chose not to think (§6).

(4) No lever that breaks loops measurably changes resolution (§7).

2 Setting and measures

The agent is Gemma 4 31B (int4, w4a16) with thinking off, in the competition's harness, with the host's sample prompts plus rules for tool-call arguments. Each task allows 4 minutes, 60 tool calls and 100 turns; sampling is T = 0.2, top-p 0.95, top-k 64. The tasks are the 109 of the competition's 129 public development tasks whose grading is sound (the reference patch passes and an empty patch fails), from fastapi, requests, httpx and rich. A run is resolved when the grader's tests pass.

Whole-task runs compare a control with three levers, two passes each unless noted: T = 1, as Google recommends (Google, 2026); a chat template with an empty thought before every model turn, as a Hugging Face pull request proposes (Hugging Face, 2026; one pass); and a proxy that collapses each run of identical tool turns (same call, same result) to its first. The harness still runs every call, but while a run lasts the model gets the same request again. Two passes of Gemma 4 12B test generality. One-step replays score the next call at recorded histories that we edit (adding copies, rewriting results, forcing a thought open): the probability of the recorded call, and one to three sampled calls per state. The histories were recorded on an earlier serving stack; where we checked, the replayed copy rate matches the recorded one (0.639 against 0.634).

A call is stale if it repeats an earlier one (same tool and arguments) with no successful file edit since, and a copy if that call came just before. A malformed call's arguments do not fit the tool. A loop task run has at least half its calls stale or malformed. After k identical calls, the hazard h(k) is the probability that the next is identical too, counted on clean steps: no budget warning in the last result and no compaction before the call. An escape, one step ahead, is a sampled call that is neither stale nor malformed. Intervals are 95%, from resampling tasks.

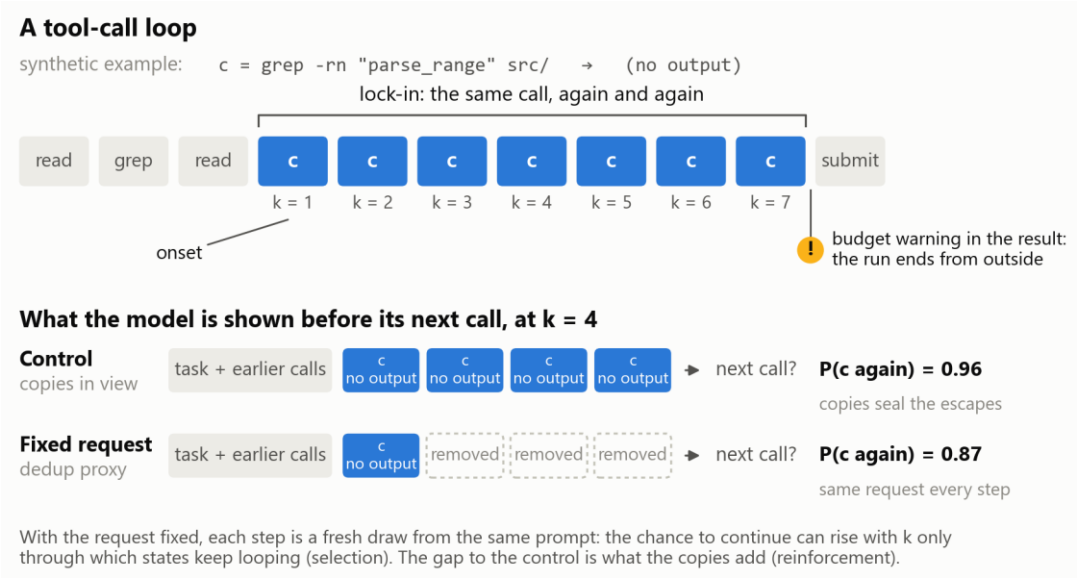
3 Loops are common and mark lost runs

In the control, 38.6% of calls are stale and 28.0% are copies, and a quarter of task runs (54 of 218) are loop task runs. Most long runs repeat a search or a command: of the 116 runs of three or more identical calls, 39 are read-only searches, 50 other commands and 26 malformed calls. Loop task runs resolve 20% of the time (11 of 54),

against 37% (61 of 164) for the rest. Within the 20 sometimes-solved tasks that have both kinds of run, the runs that loop resolve 22.8 points less often (−41.1 to −4.3). Which tasks loop is largely a property of the task: the two control passes share 15 of their 27 loop tasks, against about 7 by chance. Gemma 4 12B loops in 44.5% of its task runs. These are associations: a run that is already lost may loop because it is lost. §7 tests whether removing loops helps.

4 Selected, then sealed

In the control, a fresh call is followed by an identical one 2.4% of the time. After two identical calls the hazard is 0.68, after three or four 0.96, and from five on the agent continued at every clean step (Fig. 1b). Two mechanisms predict a curve that rises like this. Under reinforcement, each copy in context makes the next one likelier, as repetition does in text generation (Xu et al., 2022). Under selection, runs start where the model is already near certain of the call, and the longer a run lasts, the more it consists of such states.



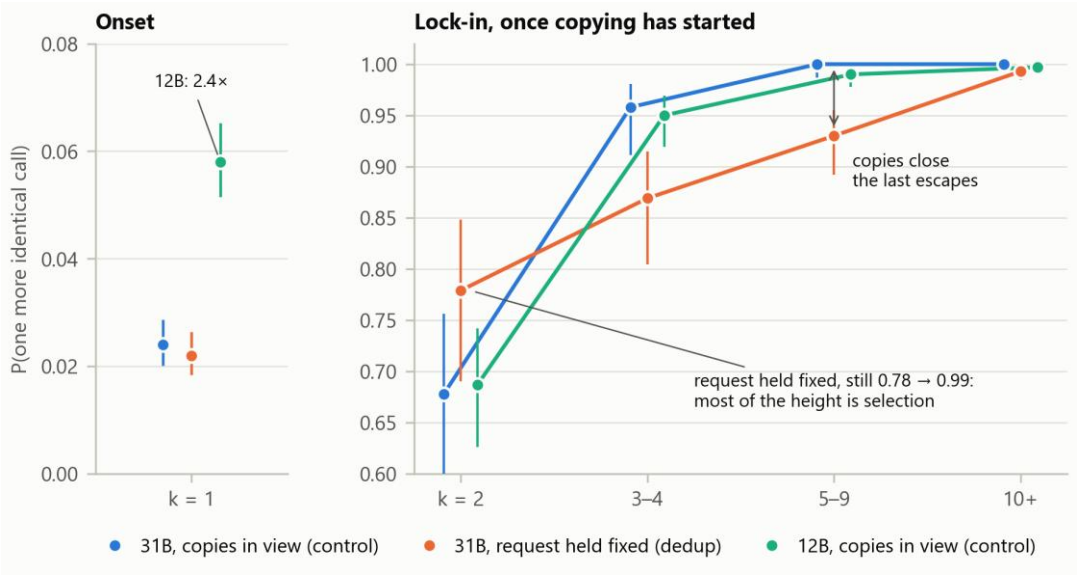


Figure 1. (a) A tool-call loop and the fixed-request test, on a synthetic call c. Before the next call at k = 4, the control shows the model every copy; the proxy collapses the run to its first call, so the request is the same at every step. (b) The hazard of one more identical call on clean steps, with the copies in view (31B and 12B) and with the request held fixed (31B). Bars are binomial 95% intervals, not clustered by task.

A rising hazard is no evidence of reinforcement by itself. Suppose each state s draws the same call again with a fixed probability p, as it does when the request is held fixed. Over the states where runs start, h(k) = E[p^k] / E[p^(k−1)], and since E[p^k]^2 ⩽ E[p^(k−1)] E[p^(k+1)] (Cauchy–Schwarz), h(k) never falls with k, whatever the mix of states. This is the frailty effect of survival analysis, where differences between individuals alone make a population's hazard rise (Vaupel et al., 1979). The proxy separates the two mechanisms: with the request fixed, any rise is selection, and the gap to the control is what the copies add (Fig. 1a).

With the request fixed, the hazard still rises: 0.78 after two identical calls, 0.87 after three or four, 0.93 after five to nine and 0.99 after ten or more. Most of the curve's height is selection. The copies add nothing measurable at k = 2 (proxy minus control, paired over tasks: +7.6 points, −8.9 to +22.4). From three calls on, they close the escapes that remain: −7.0 points (−13.3 to −0.8) at three or four, and −4.1 (−7.3 to −1.1) at five to nine. Without the copies in view the agent ends 47% of its runs of three or more itself, against 15% (+32 points, +20 to +45), yet about three quarters of its runs of ten or more persist. One step ahead, at 60 fixed histories after a search that found nothing, identical copies raise the probability of the same search from 0.002 with one copy to 0.43 with four. Re-spelled copies of it, with the same results, leave it near 0.03. The copies reinforce on their exact tokens.

Gemma 4 12B starts runs more readily: it copies a fresh call 5.8% of the time, against

2.4% (+3.2 points, +2.2 to +4.2). From k = 2 on its hazard matches the 31B's, and no difference's interval excludes zero. Size moves the onset, not the lock-in. We did not run the proxy at 12B, so selection is measured at 31B only.

5 Long runs end from outside

At T = 0.2 the agent does not end a long run itself. On clean steps it continued every run of five or more identical calls: 290 of 290 steps at five to nine calls and 1,052 of 1,052 at ten or more. Of the 58 well-formed runs of ten or more, 43 ended after the harness's budget warning ("Only 10 tool call(s) remaining …"), 14 at a context compaction and one with the task. Neither exit helps much: after the warning at most ten calls remain, and after a compaction the agent sends the same call again 30–41% of the time. With the request held fixed, long runs end the same way (31 of 54 at the warning), and Gemma 4 12B ends only 4 of its 119 runs of ten or more itself. Past five calls, only a change from outside ends a run; most often it is the warning, an instruction.

6 What cues the copy

A clearer result does not stop the copy. At 101 states where an empty search had just been repeated (23 tasks), we rewrote the repeated results in a 2 × 2 design. Varying their IDs changes the probability of the copy by −0.005 (−0.009 to −0.002), and a faithful note ("The search completed and found zero matches …") raises it slightly, by +0.020 (+0.005 to +0.036; its added tokens alone: −0.004). An instruction does move it: a controller's note that the call repeated the one before, and to try a different approach, took the probability of a third or later copy from 0.94 to 0.49, at other states. So does a break in the pattern. At the 56 copy states, ending the context with an empty thought takes it from 0.79 to 0.22; with an empty thought before every earlier call too, it returns to 0.82. The empty thought breaks the pattern; it does not switch the model's mode.

The model's own thinking does not supply that break. Free to think, it wrote a thought at 72% of second copies but at only 23% of third or later ones. At 213 looping states in 44 tasks, we forced a thought open for 32 tokens. Where the model had skipped the thought (114 states), the next call escaped 57.9% of the time, against 10.5% (+47.4 points, 33.0 to 61.1); where it had thought (99 states), forcing changed nothing (62.6% against 61.6%). The model skips thinking where a thought would help. These escapes are one step ahead (§9).

7 Breaking loops over whole tasks

Each lever changes the loops (Table 1). T = 1 cuts the stale share of calls by 18.2 points and loop task runs from 24.8% to 9.2%, yet a run once under way still continues 87% of

the time at three or more calls. The empty thought before every call makes runs nine calls shorter but more numerous, and the stale share does not fall. The proxy lets the agent end short runs itself, but the share of copies stays where it was.

Table 1. Whole-task levers against the control, Gemma 4 31B on 109 tasks unless noted. Differences are lever minus control in percentage points, paired over tasks, with 95% intervals from 2,000 task resamples. The 12B row shows the size of change the design detects.

Lever	Passes	Effect on loops	Resolved	Difference
None (control, T = 0.2)	2	stale 38.6% of calls; loop task runs 24.8%; run length 19.8 calls	72 of 218	–
T = 1	2	stale −18.2 (−24.8 to −11.0); loop task runs 9.2%	71 of 218	−0.5 (−5.0 to +4.1)
Empty thought before every call	1	runs 9 calls shorter, +0.35 runs per task; stale +1.6 (−4.5 to +7.6)	38 of 109	+1.8 (−3.7 to +7.8)
Proxy: request held fixed	2	own exits of runs of 3+ 47% vs 15%; copies −2.6 (−8.2 to +2.8)	73 of 218	+0.5 (−3.7 to +4.6)
Gemma 4 12B, no lever	2	stale +14.8 (+8.9 to +20.7); loop task runs 44.5%	24 of 218	−22.0 (−29.8 to −15.1)

None of them measurably changes resolution, and breaking a loop does not recover the task. T = 1 un-looped 46 of the control's 54 loop task runs: the control solved 10 of those tasks and T = 1 solved 9, spending the freed budget on new calls and source edits. In 23 of 24 such tasks in the first pass, the control's first stale call came before its first source edit. In the early run of §1, only 6 of the 23 failures with many repeats were solved by any of nine other agent runs, about 2 of 70 tasks. The design detects a change the size of the 12B's (−22.0 points) but not one of one or two tasks, since a single run varies by about ±2.2 tasks. An inconclusive difference is not equivalence.

8 Related work

Repetition in text. Generators repeat themselves more with each repetition in context (Holtzman et al., 2020; Xu et al., 2022), and reasoning models loop at low temperature, where raising it is “a stopgap” (Pipis et al., 2026). Our tool-call loops show the same rising curve, but holding the request fixed shows that most of it is selection, not self-reinforcement.

Agents copying themselves. Agents condition on their own errors (Sinha et al., 2026) and imitate their earlier responses (Wan et al., 2026). In models up to 1.7B, a failed call

in context makes the same call likelier, mostly through its surface form (Gumaan, 2026); we find the same exact-token effect at 31B. On screened histories, agents take a third identical action after two 37% of the time (Liu et al., 2026); ours does 68% of the time. Gemma 4 31B without thinking also loops in another harness (Yuan et al., 2026).

Repeats as symptoms. In SWE-smith, warnings, resampling and temperature bumps cut repeats while resolution fell, and the authors read repeats as "symptoms of the model's difficulty" (Yang et al., 2025). Our three levers test that reading, and none raises resolution. Fork-and-Flush escapes idea basins when fresh-context branches are scored and the best is kept (Cai et al., 2026); a coding agent has no such score during a run, and removing the copies does not help it solve more.

Detectors. OpenHands acts after four identical action–observation pairs, and Gemini CLI after five identical calls (OpenHands, 2026; Gemini CLI, 2026). At 31B, lock-in is near complete after three.

9 Limits and implications

The evidence comes from two sizes of one model family (int4), with thinking off, in one harness, on 109 tasks from four repositories. The mechanisms and the levers were measured at 31B only; the 12B tests the descriptive claims. Two passes cannot detect a change of one or two tasks. The forced thought was tested one step ahead, never over whole tasks, and an escape one step ahead need not be progress. The probabilities we report are re-draws of the same call, not calibrated confidence, and nothing here looks inside the model. The empty-thought template and the proxy test mechanisms, not entries: a competition submission can change neither the chat template nor the requests.

Three things follow for agent builders. Lock-in is near complete by the third identical call, earlier than common detectors act. Raising the temperature, breaking the pattern or removing the copies ends loops without recovering the task; our reading is that the loop marks a state the model is already sure of rather than causing the failure. A lever worth testing next acts where the model chose not to think, or brings the agent information it lacks. Breaking a loop frees budget, but it does not recover the task.

GitHub Repository

https://github.com/usp787/Gemma4_paper_track

Use of AI tools

Claude (Anthropic) helped write the experiment and analysis code, checked the references, and drafted parts of this text from the author's records. ChatGPT (OpenAI) gave advice on the experiment's design and helped interpret and analyze the experiment results. The author directed the study, revised the text, and is responsible for its content.

References

Cai et al. 2026. Fork-and-Flush: Escaping Idea Basins in Autoresearch Agents. arXiv:2610.07447.

Gemini CLI. 2026. Loop detection service (packages/core/src/services/loopDetectionService.ts, commit acae712). https://github.com/google-gemini/gemini-cli.

Google. 2026. Gemma 4 model card. https://ai.google.dev/gemma/docs/core/model_card_4.

Gumaan. 2026. Feedback That Backfires: Why Small Language Model Agents Repeat the Call They Just Watched Fail. arXiv:2608.23651.

Holtzman et al. 2020. The Curious Case of Neural Text Degeneration. ICLR. arXiv:1904.09751.

Hugging Face. 2026. google/gemma-4-31B-it, pull request #147. https://huggingface.co/google/gemma-4-31B-it/discussions/147.

Liu et al. 2026. When History Fails to Become Experience: Action Calibration in Language Agents. arXiv:2610.02769.

OpenHands. 2026. StuckDetector (openhands/controller/stuck.py, commit 6e4ac8e). https://github.com/OpenHands/OpenHands.

Pan et al. 2025. Training Software Engineering Agents and Verifiers with SWE-Gym. arXiv:2412.21139.

Pipis et al. 2026. Wait, Wait, Wait··· Why Do Reasoning Models Loop? ICML. arXiv:2512.12895.

Sinha et al. 2026. The Illusion of Diminishing Returns: Measuring Long Horizon Execution in LLMs. ICLR. arXiv:2509.09677.

Vaupel et al. 1979. The Impact of Heterogeneity in Individual Frailty on the Dynamics of Mortality. Demography 16(3), 439–454. doi:10.2307/2061224.

Wan et al. 2026. Mitigating Conversational Inertia in Multi-Turn Agents. ICML. arXiv:2602.03664.

Xu et al. 2022. Learning to Break the Loop: Analyzing and Mitigating Repetitions for Neural Text Generation. NeurIPS. doi:10.52202/068431-0223.

Yang et al. 2025. SWE-smith: Scaling Data for Software Engineering Agents. arXiv:2504.21798.

Yuan et al. 2026. Agentic AI Workload Characterization. arXiv:2605.26297.